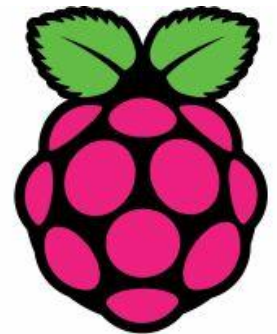


Introduction aux Systèmes d'Exploitation
Multi-Tâche et Temps Réel
Application au Système d'Exploitation LINUX
sur RaspberryPI:
Programmation Système
en Langage C



Tux



RaspberryPi

Semaine de Formation ADESFA - Douala
Patrick Bonnin 28/10 – 01/11/2024

Enseignement en Spécialité Systèmes Electroniques Embarqués

- « Résumé » très succinct en 4h d'un enseignement en Système d'Exploitation :
- 3 modules de 28h :
 - ✓ Système d'Exploitation Multi-Tâche ;
 - ✓ Système d'Exploitation Temps Réel ;
 - ✓ Système d'Exploitation Temps Réel Avancé.
- Systèmes d'Exploitation Multi-Tâche :
 - ✓ fork(), signaux, synchro, mémoire partagée, tubes ...
 - ✓ Réalisé en formation en distanciel, ressources prêtes.
- Systèmes d'Exploitation Temps Réel :
 - ✓ threads, horloges, timer, placement sur cœur, mutex ...
 - ✓ Réalisé en formation en distanciel, ressources à terminer.
- Systèmes d'Exploitation Temps Réel Avancé :
 - ✓ Priorités, ordonnancement, PIP, programmation SPMD (threads), évaluation de performances (benchmarks ...).
 - ✓ Juste démarré , ressources à terminer.

Contenu du « Résumé »

- **Introduction:**
 - ✓ Système d'Exploitation Multi-Tâche, Temps Réel ;
 - ✓ Temps Partagé / Réel Parallélisme ;
 - ✓ Intérêt.
- **Multi Application:**
 - ✓ Création de Processus : fork ();
 - ✓ Recouvrement de Processus.
- **Systèmes d'Exploitation Temps Réel :**
 - ✓ Threads,
 - ✓ Horloges temps réel ...
- **Systèmes d'Exploitation Temps Réel Avancé :**
 - ✓ Introduction à l'évaluation temporelle (Benchmarks)

Qu'est ce qu'un Système d'Exploitation

- Couche Logicielle qui remplit 2 rôles :
 - ✓ Gestion de l'Ordinateur : matériel (processeur, mémoires vive et morte (composants), disque dur, clavier, souris, écran), ainsi que la mémoire (système de fichiers) et les programmes s'exécutant.
 - ✓ Interface avec le ou les utilisateurs.
- Exemple d' « Operating Systems » connus :
 - ✓ DOS ou MS-DOS, Windows 3.1, 95, 98, Millenium, XP, Vista, 7 sur PC,
 - ✓ Mac OSX sur Macintosh d'Apple (c'est un UNIX!)
 - ✓ UNIX : gros ordinateurs : CRAY, stations de travail : Sun (Solaris, également sur PC), Silicon Graphics,
 - ✓ Temps Réel : QNX, VxWorks, RT-Linux, RTAI, RTEMS, Xenomai, PREEMPT_RT.

Historique UNIX/C/Linux

- 1969 : Système d'Exploitation Unix:Ken Thompson, Dennis Ritchie
 - ✓ Bell Telephon AT&T, en langage d'assemblage sur PDP7.
- 1970 : Langage B :Ken Thompson :
 - ✓ Compilateur, inclue les structures algorithmiques : Boucles. Test ;
 - ✓ Réécrit UNIX sur PDP11
- 1972 : Langage C : Dennis Ritchie, Brian Kernigham :
 - ✓ Inspiré du Langage B, introduit les types de données.
- 1973 : Réécriture de UNIX à 90% en Langage C sur PDP11
- 1983 à ... : Projet GNU : Richard Stallmann (MIT)
 - ✓ Chaîne d'Outils : du compilateur C à l'OS, Gnu Emacs,
 - ✓ Portabilité sur diverses architectures (ARM, etc ...)
- 1991 : Linux:
 - ✓ Projet scolaire de Linus Torvalds, Univ Helsinki, OS compatible UNIX sur PC, à partir de Minix : Andrew Tanenbaum.
 - ✓ Code diffusé sur Internet : repris par des passionnés : R.Card (syst fichiers), X.Leroy (Threads)
 - ✓ Distributions : Slackware, RedHat, Suze, Mandrake, Debian, Fedora, Ubuntu, Knoppix (CD), Cygwin (émulation Windows)

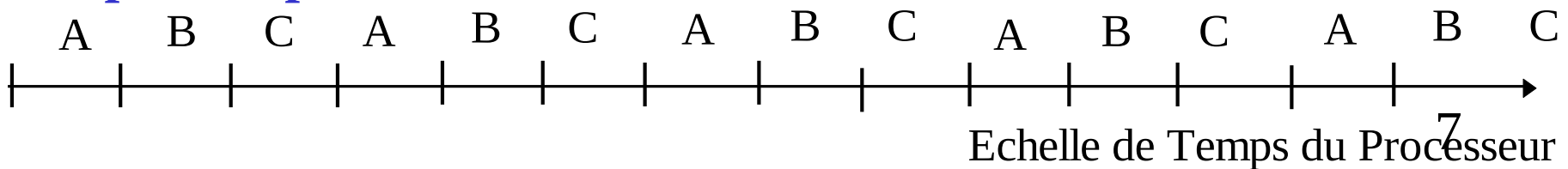


Systemes Multi Tâche et Systemes Temps Réel

- 1) Besoin : Effectuer plusieurs Tâches en Simultanéité
- Exemple : Robotique Mobile et Autonome:
 - ✓ Véhicule automatique, robot à roues, ou à pattes animaloïde ou humanoïde, en interaction avec l'homme.
 - ✓ Rôle : se déplacer en toute sécurité dans son environnement, pour effectuer une action : connaissance de l'état de l'environnement et de l'action en cours.
 - ✓ Développements Algorithmiques intégrés en 3 parties indépendantes : impossible en une seule !!!
 - « Intelligence » : gestion de l'action en cours,
 - « Prioritaire » de bas niveau : déplacement, notamment pour un robot à pattes,
 - « Acquisition de l'Information » dont « Vision »
- Fonctionnement en Simultanéité, incluant Synchronisation et Echanges d'Informations**
- 2) Exploitation de la Puissance de Calculs du Processeur :
 - ✓ Multicoeur et/ou Temps Partagé;
 - ✓ Utilisation de petites cartes : succès industriel de RaspberryPI

Notion de Temps Partagé « since 1969 »

- Le Temps Processeur : « Microscopique »
 - ✓ 2 GHz : cycle horloge = $0,5 \cdot 10^{-9}$ s, = 0,5 ns.
 - ✓ 1 instruction : environ 10 cycles horloge = 5 ns
 - ✓ Echelle du Temps Processeur : 10 ns = 0,01 μ s.
- Le Temps Humain : « Macroscopique »
 - ✓ Ordre de grandeur : 1/10s à 1 s.
- de cette différence : le Temps Partagé :
 - ✓ Plusieurs processus se partagent le même temps processeur,
 - ✓ Ils semblent s'exécuter en parallèle à notre échelle ;
 - ✓ Pseudo Parallélisme : vrai parallélisme => multi-cœur ou multi – processeur ;
 - ✓ Même avec Multi-Cœur / Processeur : Temps partagé, car plus de processus !!!



Deux types de Parallélisme

- **Temps Partagé :**
 - ✓ Depuis 1969, sur un mono-processeur.
- **Réel Parallélisme:**
 - ✓ Depuis l'apparition des multicoeurs 2000 (IBM), 2005 (Intel).
- **Co habitation des deux types :**
 - ✓ Plus de tâches que de cœurs en général !
- **Une fonctionnalité essentielle : « Load Balancing » :**
 - ✓ « Permet » de garantir à chacune des tâches l'équité d'accès aux ressources calculatoires ;
 - ✓ Pour cela, elle va gérer le partitionnement des tâches sur les différents cœurs, et les changer si nécessaire.
- **Exercice :**
 - ✓ Etudier l'algorithme du code source «BoucleInfLB.c ».
 - ✓ Le compiler, puis le lancer d'1, 2, 3, 4, 5, 6 consoles différentes;
 - ✓ Grâce à l'utilitaire « htop », visualiser l'occupation des 4 cœurs;
 - ✓ Interprétez les migrations.

Exercice

• Réaliser l'Exercice :

✓ Algorithme

Algorithme BoucleInfLB.c

Initialisation :

- AncienCoeur = Lecture_CoeurCourant()

Boucle Infinie

- NouveauCoeur = Lecture_CoeurCourant()

- Si NouveauCoeur != AncienCoeur

◦ Afficher : Migration AncienCoeur => NouveauCoeur

◦ AncienCoeur = NouveauCoeur

Exercice

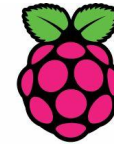
- Code Source:

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sched.h>

int main (int argc, char **argv)
{
    int oldCpu, newCpu;
    oldCpu = sched_getcpu();
    fprintf(stderr, "Nombre Cpu %d\n", sysconf(_SC_NPROCESSORS_ONLN));
    fprintf(stderr, "Processus %d de pere %d sur coeur %d\n",
        getpid(), getppid(), oldCpu );

    for (;;)
    {
        newCpu = sched_getcpu();
        if (newCpu != oldCpu)
        {
            fprintf(stderr, "Processus %d : migration %d => %d\n",
                getpid(), oldCpu, newCpu);
            oldCpu = newCpu;
        }
    }
    return EXIT_SUCCESS;
}
```

- **Limite du Temps Partagé :**
 - ✓ La Puissance du Processeur !!!
 - ✓ Plus il y a de processus « gourmands » en puissance de calcul, moins ces processus s'exécutent vite !!!
 - ⇒ Comment gagner à Tetris !!!
- **Multi-Cœur et Temps Partagé :**
 - ✓ Mêmes limitations ... puissance crête !



RaspberryPi

Programmation Parallèle Multi-Tâche

- **Multi-Tâche :**
 - ✓ Plusieurs Tâches s'exécutent en même temps, à l'échelle du temps humain ;
 - ✓ Soit : Multi-Application ou Multi-Processus ;
 - ✓ Soit : Multi-Thread (ou processus léger).
- **Processus ou Application :**
 - ✓ Programme qui s'exécute, mais qui est indépendant et possède sa propre mémoire ;
 - ✓ Si l'un des processus « plante », les autres continuent.
- **Thread ou Processus Allégé :**
 - ✓ Programme qui s'exécute également, mais dont la mémorisation appartient à un processus, car créé par le processus ;
 - ✓ Si le processus se termine, le thread se termine prématurément également.

Multi-Tâche : Application vs Thread

- Multi-Application : fork()

- ✓ Mémoires séparées, nécessite la mise en œuvre d'une mémoire partagée pour communiquer ;
- ✓ Programmation plus complexe ;
- ✓ Nécessite une MMU : Memory Management Unit, composant matériel qui n'existe que sur les micro-processeurs ;
- ✓ Réalisation de Programmes complexes nécessitant une synchronisation et / ou des échanges de données.

- Multi-Threads:

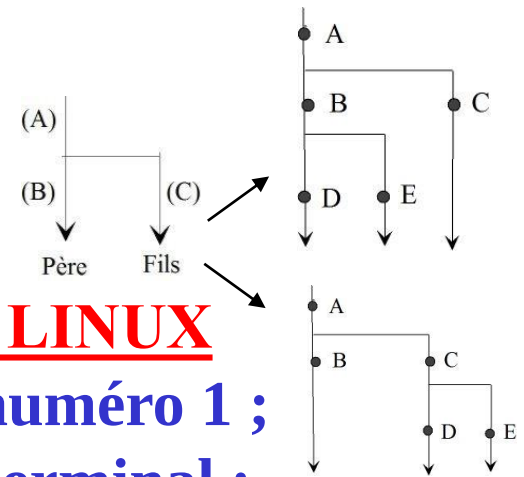
- ✓ Mémorisation du processus, donc commune aux threads, communication simplifiée ;
- ✓ Ne nécessite pas de MMU : valable sur micro-contrôleurs.

- Systèmes d'Exploitation :

- ✓ LINUX : OK sur RaspberryPI, Jetson Nano, NOK sur Arduino, supporte les deux types de Multi-Tâche ;
- ✓ freeRTOS : OK sur Arduino, une application, multi-thread.

Création de Processus sous UNIX / Linux : Théorie

- Modèle Biologique de la Division Cellulaire :**
 - ✓ Une cellule se divise en deux cellules identiques ;
 - ✓ Lors de la Création : le processus courant appelé PÈRE donne naissance à un nouveau processus identique appelé FILS ;
 - ✓ Chaque processus a un processus PÈRE ;
 - ✓ Un PÈRE peut avoir plusieurs FILS ;
 - ✓ Le FILS peut être PÈRE à son tour .
- Fonctionnement d'une Machine sous UNIX / LINUX**
 - ✓ Boot : Création du Processus « Init », de numéro 1 ;
 - ✓ « Init » crée ensuite un processus fils par terminal ;
 - ✓ Les processus sont créés en arborescence ;
 - ✓ Si un processus PÈRE disparaît, le processus FILS est rattaché au processus INIT ;
 - ✓ Lorsqu'on exécute un processus d'une console ou « shell » : le shell est le processus PÈRE.



- **Identificateur : Entier Naturel, Positif strict**
 - ✓ Les Identificateurs vont en croissant, au fur et à mesure de la création des processus ;
 - ✓ PID : Identificateur du Processus, PPID identifiant du PÈRE ;
 - ✓ Dans nos exemples simples : généralement $PID = PPID + 1$ pour le premier fils !!!
 - ✓ Arrêt d'une machine UNIX pour « maintenance » lorsque les PID deviennent très élevés.
- **PID et PPID accessibles par primitive système :**
 - ✓ Nouveau type : pid_t (int) : #include <unistd.h>
 - ✓ pid_t getpid (void); // pour l'identifiant du processus
 - ✓ pid_t getppid (void); // pour l'identifiant du processus PÈRE
 - ✓ La commande : « ps -al » permet de visualiser l'état du processus S, son identifiant PID, celui de son père PPID, ainsi que la commande CMD. « ps -all » permet également de voir l'identifiant de thread LWP ;
 - ✓ Etats possibles : run : R, bloqué S, stoppé T, zombie Z. 15

- Appel Système : int fork(void)
 - ✓ Le processus appelant le « fork() », est le processus PÈRE ;
 - ✓ L'Appel Système « fork() » crée une copie exacte du processus original : le processus FILS ;
 - ✓ Le PÈRE et le FILS ont chacun leur mémoire propre, mais identique avec les mêmes noms de variables (locales et globales) et même valeurs initiales ;
 - ✓ Si le PÈRE (resp. le FILS) modifie l'une de ses variables, la modification ne sera pas vue par le FILS (resp. le PÈRE) ;
 - ✓ Les fichiers ouverts par le PÈRE, avant la création du fils, seront accessibles au FILS dans les mêmes conditions ;
 - ✓ Les modifications des fichiers sont vues par les deux.
- Différence Fondamentale : le retour du Fork
 - ✓ Le PÈRE : l'Identifiant du Fils, nombre entier positif strict ;
 - ✓ Le FILS : « 0 » ;
 - ✓ Permet de différencier dans le code les processus PÈRE et FILS, de codes écrits dans le même fichier.

Code Père Fils

- Code Source:

```
pid_t pid, ppid, ret;
pid = getpid();
ppid = getppid();
fprintf(stderr, "(A) Pere %d de Pere %d Core %d\n",
        pid, ppid, sched_getcpu());

ret = fork(); /* creation du processus */
if (ret < 0) /* si Erreur */
{
    fprintf(stderr, "Erreur de Creation\n");
    exit(0);
}
else if (ret > 0) /* Processus Pere */
{
    fprintf(stderr, "(B) Pere %d de Pere %d ret = %d Core = %d\n",
            getpid(), getppid(), ret, sched_getcpu());
    for(;;);
    exit(0);
}
else /* Processus Fils */
{
    fprintf(stderr, "(C) Fils %d de Pere %d Core = %d\n",
            getpid(), getppid(), sched_getcpu());
    for(;;);
    exit(0);
}
```

- **Liaison entre les Processus Père et Fils :**
 - ✓ Le Père connaît le PID de son fils grâce au retour du `fork()`;
 - ✓ Le Fils connaît l'identifiant de son Père: `getppid()`;
 - ✓ Possibilité de se synchroniser par signaux :
 - Processus rapide sur processus lents (échange de données, modèle producteur / consommateur) ;
 - Fonctionnement alterné ...
- **Inconvénient de cette programmation :**
 - ✓ Duplication de la mémoire ;
 - ✓ Ecriture des codes : il faut disposer des codes source des fonctions.
- **Solution : le Recouvrement de Processus :**
 - ✓ Le remplacement d'un processus par un autre, de mêmes identifiants pid et ppid, avec une mémoire différente ;
 - ✓ Lance un autre programme exécutable, dont on ne dispose pas des sources (ex d'une bibliothèque sur étagère) ;
 - ✓ Lancement de tâches au boot : synchro, échange de données.

• Programme Recouvrant :

- ✓ Le code dont l'exécutable est dans le répertoire bin, ici affiche un mot et un entier, passés par la ligne de commande.

```
int main (int argc, char **argv)
{
    pid_t pid, ppid;
    int val;
    char Nom [100];

    if (argc != 3)
    {
        fprintf(stderr, "USAGE : Affiche mot entier \n")
        exit(0);
    }
    pid = getpid();
    ppid = getppid();
    strcpy(Nom, *++argv);
    val = atoi(*++argv);

    fprintf(stderr, "Affiche %d de Pere %d \n",
        pid, ppid);
    fprintf(stderr, "Affiche : %s %d \n", Nom, val);
    return EXIT_SUCCESS;
}
```

• Fils Recouvert :

- ✓ Permet au Père de lancer un autre programme en parallèle ;
- ✓ Changer le chemin absolu menant à la commande « Affiche »

```
ret = fork();
if (ret < 0)
{
    fprintf(stderr, "Erreur de Creation\n");
    exit(0);
}
else if (ret > 0)
{
    fprintf(stderr, "(B) Pere %d de Pere %d ret = %d\n",
        getpid(), getppid(), ret);
    sleep(5);
    fprintf(stderr, "Pere : Terminaison \n");
    exit(0);
}
else
{
    fprintf(stderr, "(C) Fils %d de Pere %d\n",
        getpid(), getppid());
    execl("/home/isty/ADESFA/SemaineDouala/bin/Affiche",
        "Affiche", "Toto", "10", (char *)NULL);
    fprintf(stderr, "NE DOIT PAS APPARAÎTRE \n");
    exit(0);
}
```

- **THREAD = Processus « Allégé »:**

- ✓ Autre moyen d'implanter du Multi-Tâche, les threads s'exécutent en parallèle ;
- ✓ Processus = Main Thread, les threads créés sont des threads secondaires . Un thread peut créer un thread ;
- ✓ Tous les threads d'un même processus partagent le même espace mémoire : la commutation entre deux threads d'un même processus est plus rapide !!
- ✓ l'Accès concurrentiel aux données globales entre threads peut provoquer des aléas de fonctionnement, en fonction de l'ordre d'accès aux données provenant des commutations :
=> Mécanismes réalisant l'Exclusion Mutuelle
- ✓ **ATTENTION : Si le processus ou Main Thread disparaît : disparition de tous les threads secondaires !!**
- ✓ Il est possible de Détacher un Thread « Indépendant », mais le problème précédent demeure !
- ✓ Bibliothèque Linux Threads de X. Leroy de l'INRIA,
- ✓ Respecte la norme POSIX 1c : compatible UNIX et QNX



- Inclure la bibliothèque : #include <pthread.h>,
- Compilation + complexe :
 - ✓ Ajouter la constante `-D_REENTRANT` à la compilation ;
 - ✓ Ajouter la bibliothèque `-lpthread` à l'Édition de liens :
`gcc -c monprog.c -D_REENTRANT`
`gcc -o monprog monprog.o -lpthread`
Ou : `gcc -o monprog monprog.c -D_REENTRANT -lpthread`
- Programmation par Threads:
 - ✓ Threads principal et secondaires : codes séparés,
 - ✓ $\neq$ `fork()`, sauf avec le Recouvrement de processus,
 - ✓ Programmation plus propre : la fonction `thread` qui va s'exécuter en parallèle est passée en paramètre par pointeur de fonction.



- Type de thread : pthread_t,
- Primitive de Création de Threads : pthread_create()
 - ✓ Prototype : `int pthread_create (pthread_t *thread, pthread_attr_t *attribut, void * (*fonction)(void *argument), void *argument);`
 - Où : thread est un pointeur sur l'identifiant du thread,
 - *attribut = NULL, attributs par défaut du thread,
 - `void *(*fonction)(void *argument)` pointeur de fonction, sur la fonction thread,
 - `void *argument` : argument à passer en paramètre à la fonction thread.
- Primitive de terminaison du thread :
 - ✓ Prototype `int pthread_exit (void *retour);`
 - ✓ **Attention** : `exit(0)` termine le thread, MAIS également le Processus ... donc tous les threads !!!



- **Primitive obligeant l'Attente du retour du thread :**

- ✓ Oblige le thread créant à attendre la terminaison du thread créé,
- ✓ Evite la disparition d'un thread secondaire, à la terminaison du thread principal,
- ✓ Prototype : `int pthread_join (pthread_t thread, void ** retour);`

Où : thread est l'identifiant du thread,

`void ** retour` : pointeur sur l'adresse de la variable ou de la variable structurée de retour.

- ✓ Mécanisme de Synchronisation par Rendez Vous entre les deux threads, pour leur disparition (idem `wait()`, `exit()`).

- **Exemple : Création d'un Thread + Synchronisation :**

- ✓ Echange de Paramètres : A la création : créant => créé, et à la terminaison : créé => créant. Passage par pointeurs génériques : « `void *` »
- ✓ Illustration de toutes les primitives précédentes !

- Fonction Thread:
 - ✓ Prototype normalisé ;
 - ✓ S'exécute en parallèle au Processus ;
 - ✓ syscall (SYS_gettid) : appel système permettant de récupérer le « TID », ie. la numérotation LINUX ;
 - ✓ pthread_self() : fonction (pthread.h) permettant de récupérer l'identifiant du thread dans cette librairie. C'est celui renvoyé au processus par la fonction pthread_create() ;

```
void *fonction_thread(void *pIn)
{
    fprintf(stderr, "Processus %d de pere %d thread %d : %ld, coeur %d\n",
        getpid(), getppid(), syscall(SYS_gettid), pthread_self(), sched_getcpu());
    sleep(10);
    fprintf(stderr, "Terminaison Thread \n");
    system("date +\"%H:%M:%S:%N\"");
    pthread_exit (NULL);
}
```

- Processus :

- ✓ Création, attente de terminaison du thread;
- ✓ Thread à « minima » : pas d'échange de paramètres.

```
void *fonction_thread(void * );
int main (int argc, char **argv)
{
    pthread_t thread; // reference un pthread
    int ret;
    fprintf(stderr, "Processus %d de pere %d thread %d sur cpu %d\n",
        getpid(), getppid(), syscall(SYS_gettid), sched_getcpu());

    if((ret = pthread_create (&thread, NULL, fonction_thread, NULL)) != 0)
    {
        fprintf(stderr, "Erreur de Creation du thread \n");
        exit (0);
    }
    fprintf(stderr, "Processus : thread cree : %ld\n", thread);

    sleep(5);
    fprintf(stderr, "Fin activite : Processus \n");
    system("date +\"%H:%M:%S:%N\"");
    pthread_join(thread, NULL);
    fprintf(stderr, "Terminaison Processus \n");
    system("date +\"%H:%M:%S:%N\"");
    return EXIT_SUCCESS;
}
```

- **3 Horloges :**
 - ✓ **REAL_TIME, PROCESS ... THREAD**
 - ✓ **à partir de l'horloge du processeur ;**
 - ✓ **Structure timespec comportant deux champs entier : tv_sec (nombre de seconde), tv_nsec (partie fractionnaire en ns);**
 - ✓ **Formule : $dTime = sec + 0.000\ 000\ 001 * nsec$**
- **REAL_TIME :**
 - ✓ **Nombre de seconde écoulé depuis le 1^{er} Janvier 1970 à la « précision » de la nano seconde ;**
 - ✓ **Prise de l'instant : permet d'obtenir des durées écoulées par différence ;**
 - ✓ **Sur Raspberry PI, non reliée à un réseau, référence fausse car pas de pile pour sauvegarder l'horloge.**
- **PROCESS CPUTIME_ID :**
 - ✓ **Temps d'exécution du processus et des threads créés .**
- **THREAD CPUTIME_ID :**
 - ✓ **Temps d'exécution du thread, ou du processus uniquement.**

• 3 Horloges :

✓ REAL_TIME, PROCESS ... THREAD

```
struct timespec ts[2], tsProc, tsThr;
double dDeb, dFin, dPeriod, dProc, dThr;

/* --- Mesure sleep () --- */
fprintf (stderr, "Debut \n");
clock_gettime(CLOCK_REALTIME, &ts[0]);
sleep(10);
clock_gettime(CLOCK_REALTIME, &ts[1]);
clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &tsProc);
clock_gettime(CLOCK_THREAD_CPUTIME_ID, &tsThr);
fprintf (stderr, "Fin \n");

dDeb = (double)ts[0].tv_sec + 0.000000001 * (double)ts[0].tv_nsec;
dFin = (double)ts[1].tv_sec + 0.000000001 * (double)ts[1].tv_nsec;
dPeriod = dFin - dDeb;
fprintf(stderr, "Mesure REALTIME sleep(10) = %.9lf s \n", dPeriod);

dProc = (double)tsProc.tv_sec + 0.000000001 * (double)tsProc.tv_nsec;
dThr = (double)tsThr.tv_sec + 0.000000001 * (double)tsThr.tv_nsec;
fprintf(stderr, "Temps Processus %.9lf Thread %.9lf sleep(10)\n",
    dProc, dThr);
```

Boucle d'Attente Active

- Résultat :
 - ✓ `REAL_TIME = 10 s ;`
 - ✓ `PROCESS = THREAD < 1ms ...` car mesure de sleep(10).
- Réalisation de Mesures Temporelles (Banchmarks) :
 - ✓ Attente Active != Attente Passive (sleep).
- Attente Active :
 - ✓ Boucle jusqu'à dépassement d'une échéance.

Initialisation :

- `compte = 0`

Boucle d'Attente :

◦ Faire

- `compte++`

- `Lecture_Horloge()`

- `calcul dateCourante`

◦ `Tant que dateCourante < dateEcheance`

- Attente Active :

✓ Permet par comptage de mesurer l'activité ;

```
long long AttenteActiveEcheance (double dFin)
{
    struct timespec ts;
    double dCur;
    long long compte = 0;
    do
    {
        compte++;
        clock_gettime(CLOCK_REALTIME,&ts);
        dCur = (double)ts.tv_sec
                + 0.000000001 *(double)ts.tv_nsec;
    } while (dCur < dFin);
    return compte;
}
```

- Processus :

✓ **Mesure des durées d'activité du Processus (incluant les Threads) et du Processus seul (« Main Thread ») ;**

```
clock_gettime(CLOCK_REALTIME,&ts);
dDeb = (double)ts.tv_sec + 0.000000001 *(double)ts.tv_nsec;
dFin = dDeb + 10.0;

if((ret = pthread_create (&thread, NULL, fonction_thread, (void*)&dFin)) != 0)
{
    fprintf(stderr, "Erreur de Creation du thread \n");
    exit (0);
}

compteP = AttenteActiveEcheance (dFin);
pthread_join(thread, NULL);

clock_gettime(CLOCK_PROCESS_CPUTIME_ID,&tsProc);
clock_gettime(CLOCK_THREAD_CPUTIME_ID,&tsThr);
dProc = (double)tsProc.tv_sec + 0.000000001 *(double)tsProc.tv_nsec;
dThr = (double)tsThr.tv_sec + 0.000000001 *(double)tsThr.tv_nsec;
fprintf(stderr, "Proc: Temps Proc %.9Lf (%LLd):Thread %.9Lf \n",
        dProc, compteP, dThr);
```